

Basic cloud deployment

COMPLETE LEARNING GUIDE

What You'll Learn

Code that runs correctly on a developer's laptop is not yet a deployed application — going to production means the app needs to survive crashes and restarts automatically, handle HTTPS, run consistently regardless of which machine it's built and run on, and be observable when something goes wrong, none of which `node server.js` in a terminal window provides. This guide covers containerizing a backend application with Docker so it runs identically everywhere, the basic pieces every production deployment needs (a process manager, a reverse proxy for HTTPS termination, health checks), a minimal continuous integration pipeline that tests and builds an application automatically, and the baseline logging/monitoring needed to know an app is actually healthy after it ships.

1. From `npm start` to Production: What Actually Changes

Running an app locally with `npm start` or `node server.js` works fine for development because a developer is present to restart it after a crash and doesn't care if it's slow to start or exposed only on localhost. Production removes that human safety net entirely: the process must restart itself automatically after a crash, must handle real concurrent traffic instead of one developer's test requests, must serve over HTTPS rather than plain HTTP, and typically needs to run identically across multiple machines behind a load balancer. Every one of the following practices exists to close one of these specific gaps.

2. Containerizing an Application with Docker

A Dockerfile describes exactly how to build a self-contained image of an application — its runtime, its dependencies, and its code — so that the same image runs identically on a developer's machine, in CI, and in production, eliminating the classic 'works on my machine' problem caused by differing OS versions or globally installed dependencies. Using a specific, pinned base image (`node:20-alpine` rather than a floating `node:latest`) keeps builds reproducible over time, and copying package files before the rest of the source code lets Docker cache the dependency-install layer, meaningfully speeding up rebuilds when only application code changes.

DOCKERFILE

```
FROM node:20-alpine

WORKDIR /app

COPY package*.json ./
RUN npm ci --omit=dev

COPY . .

ENV NODE_ENV=production
EXPOSE 3000

HEALTHCHECK --interval=30s --timeout=3s \
  CMD wget -qO- http://localhost:3000/health || exit 1

CMD ["node", "server.js"]
```

3. Health Checks: Letting Infrastructure Know You're Alive

A health-check endpoint (commonly GET /health, returning 200 OK with a small JSON payload as long as the app and its critical dependencies like the database connection are working) is what load balancers, container orchestrators, and uptime monitors use to decide whether an instance should keep receiving traffic. Without one, an orchestrator has no reliable way to tell a genuinely broken instance (crashed, deadlocked, unable to reach its database) from a healthy one, and will happily keep routing user traffic to a container that can't actually serve it. A good health check verifies real dependencies (can I reach the database?) rather than trivially returning 200 unconditionally.

4. Reverse Proxies and HTTPS Termination

Production traffic almost never hits an application server directly — it passes first through a reverse proxy or managed load balancer (nginx, a cloud provider's load balancer, or a CDN) that terminates HTTPS (handling the TLS certificate and encryption/decryption) and forwards plain HTTP to the application server behind it. This keeps certificate management out of application code entirely, allows one proxy to route to multiple backend services or instances, and is typically also where response compression, basic rate limiting, and static asset caching are configured, rather than duplicating that logic inside the application itself.

5. A Minimal CI Pipeline

Continuous integration automatically runs an application's tests and build on every push, catching regressions before they reach production instead of relying on a developer to remember to run tests locally. A minimal pipeline checks out the code, installs dependencies with a locked, reproducible install (npm ci rather than npm install, which respects the exact versions in the lockfile), runs the test suite, and builds the application — failing the pipeline (and blocking a merge or deploy) if any step fails.

YAML

```
name: CI
on:
  push:
    branches: [main]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
      - run: npm ci
      - run: npm test
      - run: npm run build
```

6. Injecting Secrets and Config at Deploy Time, Not Build Time

A Docker image should be built once and deployed unchanged across environments, which means environment-specific configuration (database URLs, API keys) must never be baked into the image itself — they should be injected as environment variables at container startup by the hosting platform (a cloud run service's configuration, a Kubernetes secret mounted as an env var, a PaaS's environment variable settings). Baking a secret into an image is a real security exposure: anyone who can pull that image (from a registry, from a leaked build artifact) can extract the secret, and the same image can no longer be safely reused across environments with different credentials.

- Build one image; inject different config/secrets per environment at deploy/runtime, not at build time.
- Never COPY a real .env file with production secrets into a Docker image.
- Cloud platforms (AWS ECS, Google Cloud Run, Azure App Service, Heroku, Render, Fly.io) all provide a runtime environment-variable configuration mechanism for exactly this purpose.

7. Logging and Basic Monitoring After Deployment

Once an application is running in production, print statements to a terminal a developer is watching are no longer useful — logs need to go somewhere durable and searchable, typically stdout/stderr captured by the container runtime and forwarded to a log aggregation service (CloudWatch, Datadog, or a self-hosted stack like the ELK stack). Structured logging (emitting JSON log lines with consistent fields like timestamp, request ID, and severity level, rather than free-form text) makes those logs searchable and filterable at scale. Basic monitoring (uptime checks, error-rate alerts, and dashboards for response time and request volume) is what turns 'the app is probably fine' into an actual, verifiable fact.

Common Pitfalls

- Running an application with a bare `node server.js`/`npm start`` in production with no process manager, so the app simply stays down after any crash instead of restarting automatically.
- Forgetting to set `NODE_ENV=production` (or the equivalent), leaving the app running in a slower, more verbose development mode in production.
- Baking real secrets or environment-specific config directly into a Docker image instead of injecting them at runtime, exposing them to anyone who can pull the image.
- Not exposing a real health-check endpoint, leaving load balancers/orchestrators unable to detect and route around a genuinely broken instance.
- Using a floating base image tag (`node:latest`) instead of a pinned version, causing builds to silently change behavior over time.
- Deploying without any centralized logging or monitoring, discovering an outage only when a user reports it instead of from an alert.

Key Takeaways

- Production requires automatic crash recovery, HTTPS, and consistent environments — none of which a locally run development server provides on its own.
- Docker packages an application with its exact runtime and dependencies, so the same image behaves identically across every environment it runs in.
- A real health-check endpoint (verifying actual dependencies, not just returning 200 unconditionally) is what lets infrastructure detect and route around a broken instance.
- A reverse proxy or managed load balancer typically handles HTTPS termination, keeping certificate management out of application code.
- Build one image and inject environment-specific configuration and secrets at deploy/runtime — never bake secrets into the image itself.
- Centralized, structured logging and basic uptime/error monitoring are what turn 'it's probably fine' into a verifiable fact after deployment.

Learn More

Docker Documentation — Dockerfile Best Practices

<https://docs.docker.com/build/building/best-practices/>

The Twelve-Factor App

<https://12factor.net/>

GitHub Actions Documentation

<https://docs.github.com/en/actions>